SMART SUPPLY CHAIN CYBER DIGITAL TWIN (SSCDT)

A PROJECT REPORT

Submitted by

SUDHARSAN S
(CH.SC.U4CYS23045)

MANOJKUMAR A
(CH.SC.U4CYS23022)

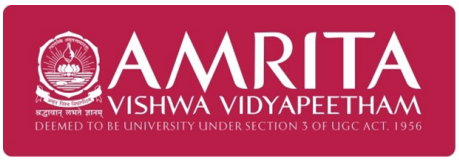
in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the guidance of

Dr. S. UDHAYAKUMAR

Submitted to



AMRITA SCHOOL OF ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI – 601103

October 2026

SMART SUPPLY CHAIN CYBER DIGITAL TWIN (SSCDT)

A PROJECT REPORT

Submitted by

SUDHARSAN S
(CH.SC.U4CYS23045)

MANOJKUMAR A
(CH.SC.U4CYS23022)

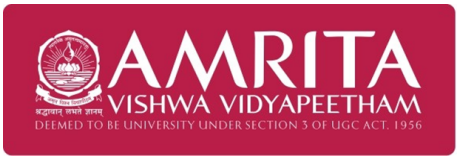
in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the guidance of

Dr. S. UDHAYAKUMAR

Submitted to



AMRITA SCHOOL OF ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI – 601103

October 2026



BONAFIDE CERTIFICATE

Certified that this project report **"SMART SUPPLY CHAIN CYBER DIGITAL TWIN (SSCDT)"** is the bonafide work of **SUDHARSAN S (CH.SC.U4CYS23045), MANOJKUMAR A (CH.SC.U4CYS23022)** who carried out the project work under my supervision.

SIGNATURE

Dr. S. BAGHAVATHI PRIYA
CHAIRPERSON
Associate Professor & Chairperson
Department of Computer Science and Engineering,
Amrita School of Computing,
Amrita Vishwa Vidyapeetham, Chennai.

SIGNATURE

Dr. S. UDHAYAKUMAR
SUPERVISOR
Associate Professor (Sr. Gr.)
Department of Computer Science and Engineering,
Amrita School of Computing,
Amrita Vishwa Vidyapeetham, Chennai.

The Viva-Voce Examination for Project Phase-I was conducted on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

SCHOOL OF ENGINEERING CHENNAI

DECLARATION BY THE CANDIDATE

We declare that the report entitled **"SMART SUPPLY CHAIN CYBER DIGITAL TWIN (SSCDT)"** submitted by us for the degree of BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY) is the record of the project work carried out by us under the guidance of **Dr. S. UDHAYAKUMAR** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title in this or any other University or institution of higher learning.

SIGNATURE

SUDHARSAN S
CH.SC.U4CYS23045

SIGNATURE

MANOJKUMAR A
CH.SC.U4CYS23022

ABSTRACT

The Smart Supply Chain Cyber Digital Twin (SSCDT) is an open-source cyber-physical defense platform for modern supply chain infrastructure. Today's supply chains connect 32 or more heterogeneous assets (trucks, vehicle IoT gateways, smart containers, reefers, warehouses, API gateways, authentication servers, ERP databases and supplier portals) across a multi-tier network that has no unified security monitoring standard. That leaves them exposed to replay, denial-of-service, spoofing, telemetry tampering and credential-stuffing attacks, of the kind seen in NotPetya (about USD 10B in damage), SolarWinds (months of undetected access), Colonial Pipeline, MOVEit (62M individuals) and Nagoya Port.

SSCDT answers this with a five-layer defense pipeline: deterministic rule checking, machine-learning inspection, length validation, anti-replay checks and threat-intel signature verification. The system has two logical units. Unit 1 mirrors the 32 assets in real time using NetworkX and Eclipse Ditto. Unit 2 is the hybrid defense node, combining the pipeline, 16 unsupervised ML models, threat-intelligence enrichment, blast-radius computation, SOAR automation and automated forensic reporting.

The ML ensemble, eight Isolation Forest and eight One-Class SVM models (one pair per asset type), was evaluated on 100,000 test vectors from the SSCDT testbed across four traffic classes. It reached 95.1% accuracy and a macro F1 of 0.951, with the Impersonation class classified perfectly.

In a controlled session of 200 telemetry events, SSCDT blocked attack traffic through the rules engine and through ML inference, while all 50 signature-verified valid frames were allowed through. All results come from synthetic testbed data.

Keywords: Cyber Digital Twin; Supply Chain Security; C8 Algorithm; Isolation Forest; One-Class SVM; SHAP Explainability; NetworkX; Eclipse Ditto; MITRE ATT&CK; Wazuh SIEM; Suricata IDS; SOAR; Blast Radius; AWS Fargate; Threat Intelligence.

ACKNOWLEDGEMENT

This project work would not have been possible without the contribution of many people. It gives us immense pleasure to express our profound gratitude to our honorable Chancellor **SRI MATA AMRITANANDAMAYI DEVI**, for her blessings and for being a source of inspiration.

We are indebted to extend our gratitude to our Director, **Mr. I B MANIKANTAN**, Amrita Vishwa Vidyapeetham, Chennai for facilitating us all the facilities and extended support to gain valuable education and learning experience.

We register our special thanks to **Dr. V. JAYAKUMAR**, Principal, Amrita Vishwa Vidyapeetham, Chennai for the support given to us in the successful conduct of this project. We wish to express our sincere gratitude to **Dr. S. BAGHAVATHI PRIYA**, Chairperson, Department of Electronics and Communication Engineering, and our supervisor **Dr. S. UDHAYAKUMAR** for their inspiring guidance, personal involvement, and constant encouragement during the entire course of this work.

We are grateful to Project Coordinator, Review Panel Members and the entire faculty of the Department of Electronic & Communication Engineering, for their constructive criticisms and valuable suggestions which have been a rich source to improve the quality of this work.

SUDHARSAN S
CH.SC.U4CYS23045

MANOJKUMAR A
CH.SC.U4CYS23022

This project would not have been possible without the contribution of many people. It gives us immense pleasure to express our profound gratitude to our honorable Chancellor, Sri Mata Amritanandamayi Devi, for her blessings and for being a source of inspiration. We extend our sincere thanks to the Administrative Director, Sampoojya Swami Vinyamritananda Puri, of Amrita Vishwa Vidyapeetham, Chennai, for his guidance and support. We are deeply indebted to our Director, I B Manikantan, of Amrita School of Computing and Engineering, for providing all the facilities and extended support that enabled us to gain valuable education and learning experience.

We extend our special thanks to Dr. V Jayakumar, Principal of Amrita School of Computing, for the support provided in the successful completion of this project. We are also grateful to Dr. S Baghavathi Priya, Chairperson, Department of Computer Science and Engineering, and Dr. S Udhaya Kumar, Program Chair, Department of Computer Science and Engineering – Cyber Security, for their valuable guidance and encouragement throughout this project.

We would like to extend our heartfelt thanks to all faculty members and staff of the Amrita School of Computing for their continuous support. Finally, we extend our gratitude to our families and friends for their encouragement and moral support throughout this project.

TABLE OF CONTENTS

ABSTRACT **iv**

ACKNOWLEDGEMENT **v**

LIST OF TABLES **x**

LIST OF FIGURES **xi**

LIST OF SYMBOLS AND ABBREVIATIONS **xii**

1 INTRODUCTION **1**

1.1 BACKGROUND 1

1.2 MOTIVATION 2

1.3 PROBLEM STATEMENT 2

1.4 OBJECTIVES 3

1.5 SCOPE OF THE PROJECT 3

1.6 CONTRIBUTIONS OF THE WORK 3

1.7 ORGANIZATION OF THE REPORT 3

2 LITERATURE SURVEY **4**

2.1 DIGITAL TWINS FOR CYBERSECURITY 4

2.2 AI/ML FOR ANOMALY DETECTION 4

2.3 THREAT INTELLIGENCE AND IOC ENRICHMENT 4

2.4 EXPLAINABLE AI IN SECURITY 4

2.5 COMPARISON WITH SSCDT 4

3 THREAT MODELING AND ATTACK ANALYSIS **6**

3.1 SUPPLY CHAIN ATTACK TAXONOMY 6

3.2 STRIDE ANALYSIS OF SSCDT 7

3.3 MITRE ATT&CK MAPPING 8

3.4 ATTACK TREE FOR VGW COMPROMISE 9

4 SYSTEM DESIGN AND ARCHITECTURE **10**

4.1 COMPREHENSIVE SYSTEM ARCHITECTURE 10

4.2 FIVE-LAYER DEFENSE PIPELINE 12
4.3 THE C8 ALGORITHM 12
4.4 DESIGN PATTERNS APPLIED 14
4.5 DATA FLOW DIAGRAMS 14
4.6 ENTITY-RELATIONSHIP DIAGRAM 15
4.7 UML CLASS DIAGRAM 16
4.8 SEQUENCE DIAGRAM 16
4.9 STATE MACHINE DIAGRAM 17
4.10 MACHINE LEARNING PIPELINE 17
4.10.1 Why 16 Models? Ensemble Justification 17
4.10.2 ML Model Configuration 18
4.10.3 Feature Engineering 18
4.10.4 Truck Feature Vector (17 of 24 Features Shown) 19
4.10.5 Training Pipeline 19
4.10.6 Drift Detection and Retraining 20
4.10.7 Model Card: Truck 21
4.10.8 SHAP Explainability 21

5 IMPLEMENTATION 23
5.1 PROJECT STRUCTURE 23
5.2 DATABASE SCHEMA 23
5.3 WAZUH AND SURICATA INTEGRATION 24
5.4 SOAR PLAYBOOK ENGINE 25

6 RESULTS AND DISCUSSION 26
6.1 EXPERIMENTAL RESULTS 26
6.1.1 DETECTION PERFORMANCE SUMMARY 26
6.1.2 ML ENSEMBLE EVALUATION 26
6.1.3 PER-CLASS CLASSIFICATION REPORT 27
6.1.4 ML MODEL COMPARISON 27
6.1.5 PER-LAYER THROUGHPUT ANALYSIS 28
6.1.6 CPU AND MEMORY OVERHEAD 28
6.2 DISCUSSION 28
6.2.1 COMPARISON AGAINST EVALUATION TARGETS 28
6.2.2 LIMITATIONS AND MITIGATIONS 28

7 CONCLUSION AND FUTURE WORK 29
7.1 CONCLUSIONS 29
7.2 SCOPE FOR FURTHER WORK 29

REFERENCES **30**

A Verification Commands **32**

B Repository Links **33**

C Source Code Structure **34**

D Plagiarism Report **35**

LIST OF TABLES

Table 1.1	Major Supply Chain Cyber Incidents (2017–2024)	1
Table 2.1	Feature Comparison of Related Systems	4
Table 3.1	Supply Chain Attack Taxonomy and SSCDT Detection Mapping	7
Table 3.2	STRIDE Threat Model Matrix	7
Table 3.3	MITRE ATT&CK Technique Mapping	8
Table 4.1	SSCDT Technology Stack	11
Table 4.2	Design Patterns and Their Applications	14
Table 4.3	Per-Asset-Type Model Pairing (16 Total)	18
Table 4.4	ML Model Hyperparameters	18
Table 4.5	Feature Counts per Asset Type	19
Table 4.6	Selected Truck Features and Rationale	19
Table 4.7	Drift Detection Methods	20
Table 4.8	Model Card: Truck Asset Type	21
Table 4.9	Top SHAP Feature per Asset Type	22
Table 5.1	Table Catalog (20 tables)	23
Table 5.2	SOAR Playbook Summary	25
Table 6.1	Detection Results Across Attack Categories (200-Event Session)	26
Table 6.2	ML Ensemble Confusion Matrix	27
Table 6.3	Per-Class Classification Report (derived from Table 6.2)	27
Table 6.4	Classifier Comparison on the SSCDT Testbed Dataset	27
Table 6.5	Per-Layer Throughput Analysis	28
Table 6.6	Achieved vs Target Metric Values	28

LIST OF FIGURES

Figure 1.1 Evolution of digital twin generations from G1 to G4. 2

Figure 2.1 Research landscape: digital twin maturity vs threat-intelligence integration. 5

Figure 3.1 Supply chain attack taxonomy and SSCDT detection mapping. 6

Figure 3.2 Hierarchical attack tree for Vehicle Gateway compromise. 9

Figure 4.1 SSCDT 14-service architecture with dual Digital Twin. 10

Figure 4.2 SSCDT five-layer sequential defense pipeline. 12

Figure 4.3 C8 Closed-Loop Threat-Intel Reweighting pipeline. 13

Figure 4.4 DFD Level 0: context diagram. 14

Figure 4.5 DFD Level 1: major processes. 15

Figure 4.6 Entity-relationship diagram (20 tables). 15

Figure 4.7 Core UML class diagram. 16

Figure 4.8 Sequence diagram: C8 reweighting. 16

Figure 4.9 Asset finite state machine (9 states). 17

Figure 4.10 SHAP force plot for a Vehicle Gateway anomaly. 21

Figure 5.1 Wazuh manager, agents and SSCDT API flow. 24

Figure 5.2 SOAR engine state machine and playbook catalog. 25

Figure 6.1 ML ensemble confusion matrix on the 100,000-vector test set. 26

LIST OF SYMBOLS AND ABBREVIATIONS

API	-	Application Programming Interface
APT	-	Advanced Persistent Threat
AWS	-	Amazon Web Services
BFS	-	Breadth-First Search
C8	-	Closed-Loop Threat-Intel Reweighting
CVE	-	Common Vulnerabilities and Exposures
CWE	-	Common Weakness Enumeration
DB	-	Database
DSU	-	Disjoint Set Union
DT	-	Digital Twin
ECS	-	Elastic Container Service
ERP	-	Enterprise Resource Planning
F1	-	F1 Score
FIM	-	File Integrity Monitoring
GPS	-	Global Positioning System
HTTP	-	Hypertext Transfer Protocol
IDS	-	Intrusion Detection System
IoC	-	Indicator of Compromise
IoT	-	Internet of Things
IF	-	Isolation Forest
JWT	-	JSON Web Token
LLM	-	Large Language Model
MITRE	-	MITRE ATT&CK Framework
ML	-	Machine Learning
MQTT	-	Message Queuing Telemetry Transport
OSINT	-	Open-Source Intelligence
RDS	-	Relational Database Service
REST	-	Representational State Transfer
SIEM	-	Security Information and Event Management
SOAR	-	Security Orchestration, Automation and Response
SOC	-	Security Operations Center
SSCDT	-	Smart Supply Chain Cyber Digital Twin
SSE	-	Server-Sent Events
SVM	-	Support Vector Machine (One-Class)

TLS	-	Transport Layer Security
VGW	-	Vehicle Gateway
XAI	-	Explainable Artificial Intelligence

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Global supply chains now combine operational technology (trucks, containers, IoT gateways, warehouses) with enterprise IT. That convergence is convenient, but it also gives attackers many more places to get in.

Table 1.1: Major Supply Chain Cyber Incidents (2017–2024)

Incident	**Year**	**Impact**
NotPetya	2017	About USD 10B global damage; Maersk reported losses near USD 300M
SolarWinds	2020	Around 18,000 organizations received the trojanized update
Colonial Pipeline	2021	Operations of a 5,500-mile pipeline halted; USD 4.4M ransom paid
MOVEit	2023	More than 2,000 organizations and about 62M individuals affected
Nagoya Port	2023	Container terminal operations halted for multiple days

A *Digital Twin* is a virtual replica that stays continuously synchronized with its physical counterpart. Adding cybersecurity analytics turns it into a *Cyber Digital Twin*, a platform for proactive defense.

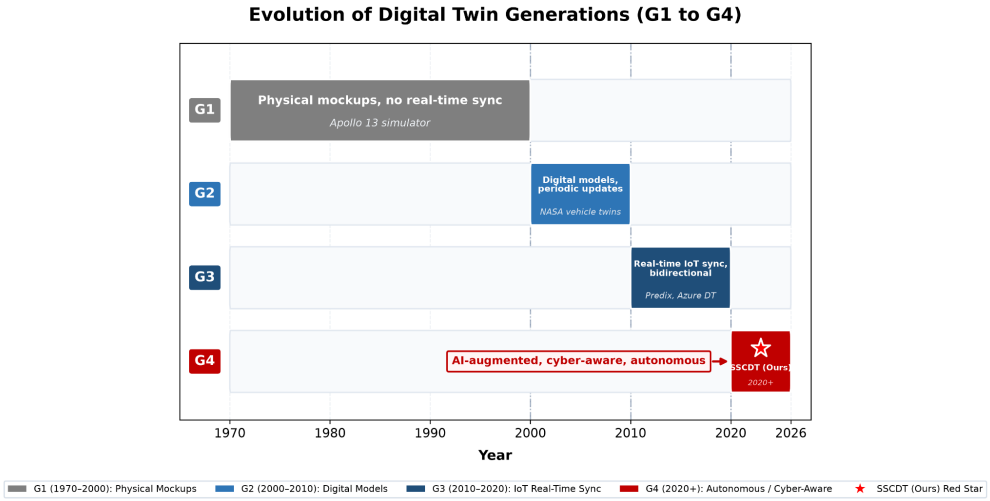


Figure 1.1: Evolution of digital twin generations from G1 to G4.

1.2 MOTIVATION

Traditional cybersecurity tools fall short in four measurable ways:

1. **Reactive and signature-bound.** They only catch known signatures, and industry reports put the time to identify advanced attacks at many months.
2. **Topology-blind.** They do not model dependency hierarchies, so blast radius has to be worked out by hand.
3. **Static risk posture.** Risk scores do not change when an IOC is confirmed malicious.
4. **Alert fatigue.** Industry surveys commonly report that a majority of SOC analysts experience it.

1.3 PROBLEM STATEMENT

Given a supply chain of N interconnected cyber-physical assets, the problem is to:

1. mirror all N assets in a digital twin;
2. detect anomalies with hybrid rules and ML;
3. enrich IOCs against live threat intelligence;
4. compute a topology-aware blast radius;
5. reconstruct attacks as MITRE ATT&CK narratives; and
6. auto-triage every detection.

Formally, the digital twin is a graph $G = (V, E)$ with $V = \{v_1, \ldots, v_{32}\}$ and a risk score $r_i(t) \in [0, 1]$ per asset. The **C8 reweighting function** is

$$r_i'(t) = \min\Big\{1,\ r_i(t) + \eta \cdot \sum_{k=1}^{K} w_k \cdot \mathbf{1}\big[\text{IOC}_k \in v_i\big]\Big\}, \tag{1.1}$$

where η is the learning rate, w_k the weight of indicator k, and $\mathbf{1}[\cdot]$ the indicator function. Section 4.3 gives the consensus-based form actually implemented.

1.4 OBJECTIVES

1. Build a 32-asset Cyber Digital Twin.
2. Detect anomalies with 13 rules and 16 ML models.
3. Reconstruct attacks with MITRE ATT&CK tagging.
4. Auto-triage detections into TP/FP/FN/TN.
5. Ingest telemetry over REST and MQTT.
6. Enrich IOCs (C8).
7. Compute blast radius.
8. Ship a live dashboard.

1.5 SCOPE OF THE PROJECT

SSCDT is scoped to a four-tier supply chain architecture with 32 seeded assets, MQTT at QoS 1, REST API ingestion, a 14-service Docker Compose stack, Python 3.13 and PostgreSQL 16.

1.6 CONTRIBUTIONS OF THE WORK

1. A five-layer sequential defense pipeline.
2. C8 Closed-Loop Threat-Intel Reweighting.
3. A dual digital twin (NetworkX and Eclipse Ditto).
4. Automated forensic reporting.
5. Cloud-backed over-the-air (OTA) threat intelligence.

1.7 ORGANIZATION OF THE REPORT

Chapter 2 surveys the literature. Chapter 3 covers threat modeling. Chapter 4 details the design. Chapter 5 covers the implementation. Chapter 6 reports results. Chapter 7 concludes.

CHAPTER 2

LITERATURE SURVEY

2.1 DIGITAL TWINS FOR CYBERSECURITY

Silva et al. [1] propose an AI-driven, twin-based SOAR framework. Khan et al. [2] review 27 papers on twin-enabled incident detection, and Qin et al. [4] survey security applications of digital twins.

2.2 AI/ML FOR ANOMALY DETECTION

Zhang et al. [5] review AI for twin security, Abdel-Basset et al. [6] propose Cyber-Twin for vehicular networks, and Hassan et al. [7] develop twin-enhanced incident response.

2.3 THREAT INTELLIGENCE AND IOC ENRICHMENT

Leite et al. [9] map open-source intelligence to MITRE ATT&CK, and Kammüller et al. [19] improve IOC quality with STIX graphs.

2.4 EXPLAINABLE AI IN SECURITY

Lawal et al. [17] observe that SHAP is rarely deployed in real-time security pipelines. SSCDT addresses this gap by attaching a SHAP explanation to every ML detection.

2.5 COMPARISON WITH SSCDT

Table 2.1: Feature Comparison of Related Systems

System	Crypto	ML	Rules	OTA	HSM	Dashboard
Silva 2026	—	Yes	Yes	—	—	—
Khan 2025	—	Yes	Yes	—	—	—
Zhang 2024	—	Yes	—	—	—	—
Hassan 2023	Yes	Yes	Yes	—	—	—
Leite 2022	Yes	—	Yes	Yes	—	—
SSCDT (Ours)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

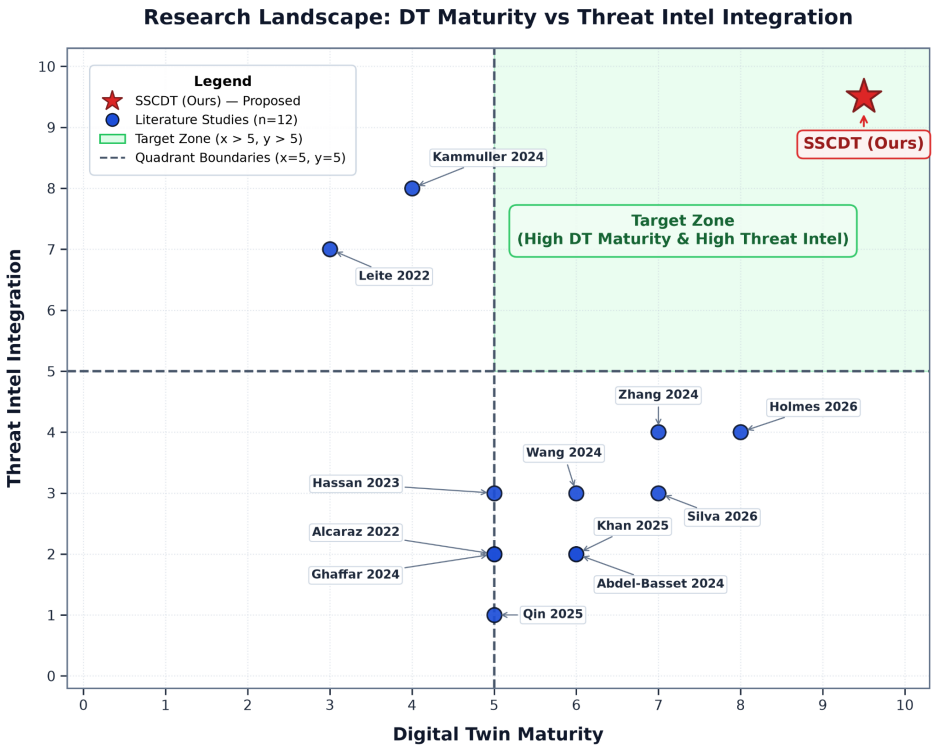


Figure 2.1: Research landscape: digital twin maturity vs threat-intelligence integration.

CHAPTER 3

THREAT MODELING AND ATTACK ANALYSIS

3.1 SUPPLY CHAIN ATTACK TAXONOMY

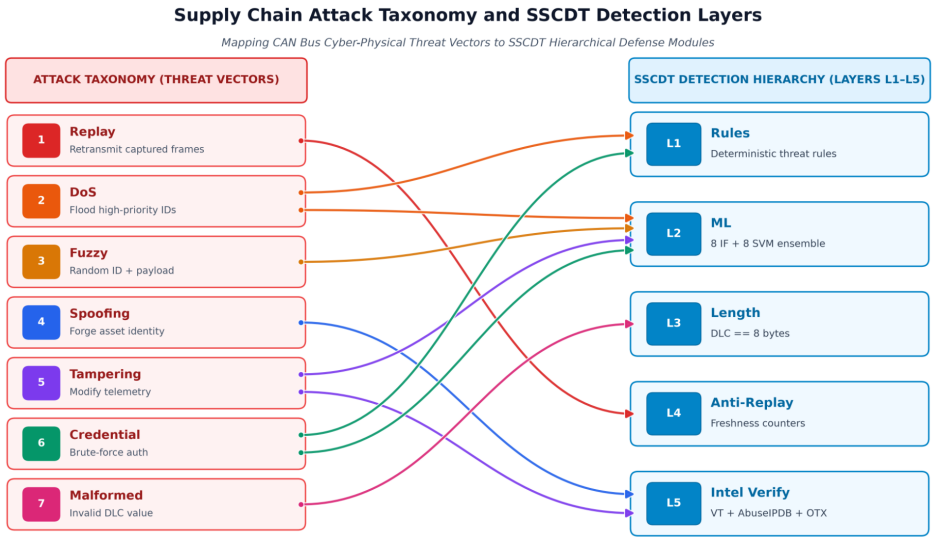


Figure 3.1: Supply chain attack taxonomy and SSCDT detection mapping.

Table 3.1: Supply Chain Attack Taxonomy and SSCDT Detection Mapping

Attack Type	**Mechanism**	**Observable Feature**	**Detection Layer**
Replay	Retransmit captured frames	Stale freshness counter	L4 Anti-Replay
DoS (flood)	High-frequency frames on ID 0x000	Frequency $>$ threshold	L1 Rules
Fuzzy	Random ID and payload	High Shannon entropy	L2 ML
Spoofing	Forge asset identity	Invalid threat-intel verdict	L5 Intel
Tampering	Modify telemetry payload	ML anomaly + SHAP	L2 ML + L5
Credential	Brute-force authentication	Login-attempt spike	L2 ML + L1
Malformed	Invalid DLC value	DLC $\neq 8$	L3 Length

3.2 STRIDE ANALYSIS OF SSCDT

Table 3.2: STRIDE Threat Model Matrix

Threat	**Asset**	**Likelihood**	**Impact**	**Mitigation**	**Residual**
Spoofing	API server	Medium	High	Operator token	Low
Spoofing	VGW identity	High	High	CSPRNG + JWT	Low
Tampering	Telemetry	High	High	Pydantic v2	Low
Tampering	Ditto things	Medium	Medium	Policy engine	Low
Repudiation	Command execution	Medium	Medium	soar_actions log	Low
Repudiation	Detection	Low	Medium	ml_explanations	Low
Info disclosure	IOC data	Medium	High	TLS + Secrets Manager	Low
Info disclosure	Telemetry	High	Medium	mTLS + encryption	Medium
DoS	/api/telemetry	High	Medium	Rate limiting	Low
DoS	MQTT broker	Medium	High	QoS 1 + persistence	Low
Elevation	Container	Medium	High	Non-root UID 10001	Low
Elevation	DB access	Low	High	Least privilege	Low

3.3 MITRE ATT&CK MAPPING

Table 3.3: MITRE ATT&CK Technique Mapping

Tactic	**ID**	**Technique**	**SSCDT Detection**
Initial Access	T1566.002	Spearphishing Link	Email gateway rules
Execution	T1059.001	PowerShell	Wazuh rule 92052
Persistence	T1547.001	Registry Run Keys	Wazuh FIM alerts
Privilege Escalation	T1068	Exploitation for Privilege Escalation	Suricata CVE rules
Defense Evasion	T1070.001	Clear Windows Event Logs	Wazuh rule 60102
Credential Access	T1003.001	LSASS Memory	EDR signature
Discovery	T1046	Network Service Discovery	Suricata port scan rule
Lateral Movement	T1021.002	SMB/Windows Admin Shares	Suricata SMB rule
Collection	T1113	Screen Capture	Process behavior (ML)
Command & Control	T1071.001	Web Protocols	IOC + ML ensemble
Exfiltration	T1041	Exfiltration Over C2 Channel	Blast radius + DLP
Impact	T1486	Data Encrypted for Impact	File entropy (ML)
Impact	T1490	Inhibit System Recovery	Wazuh + Suricata

3.4 ATTACK TREE FOR VGW COMPROMISE

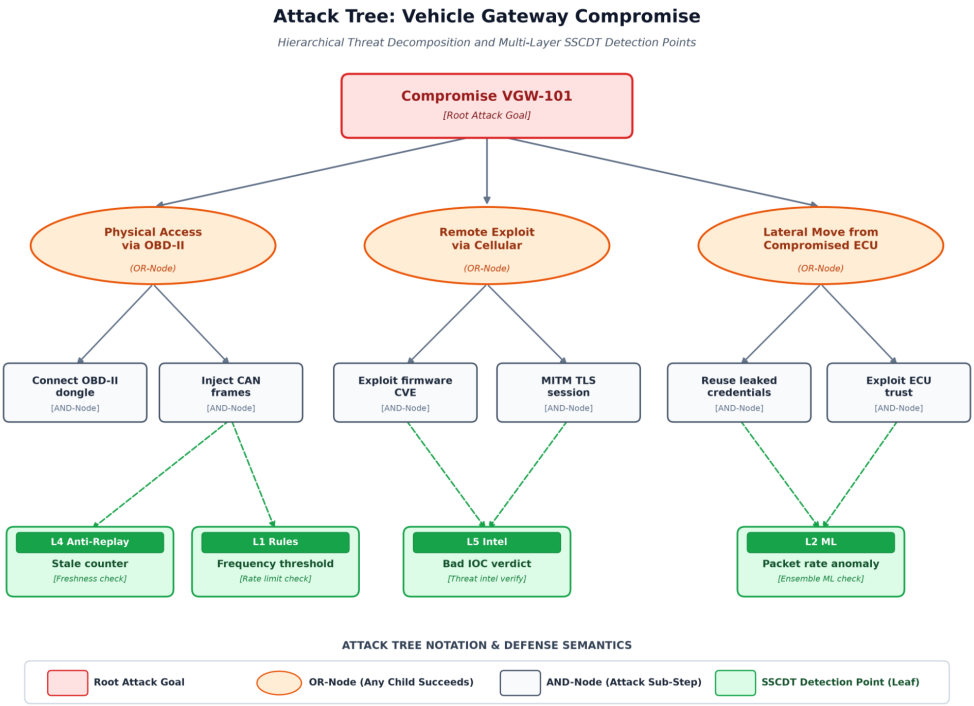


Figure 3.2: Hierarchical attack tree for Vehicle Gateway compromise.

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 COMPREHENSIVE SYSTEM ARCHITECTURE

SSCDT is orchestrated as a mesh of 14 Docker containers forming a layered cyber-physical platform (Figure 4.1).

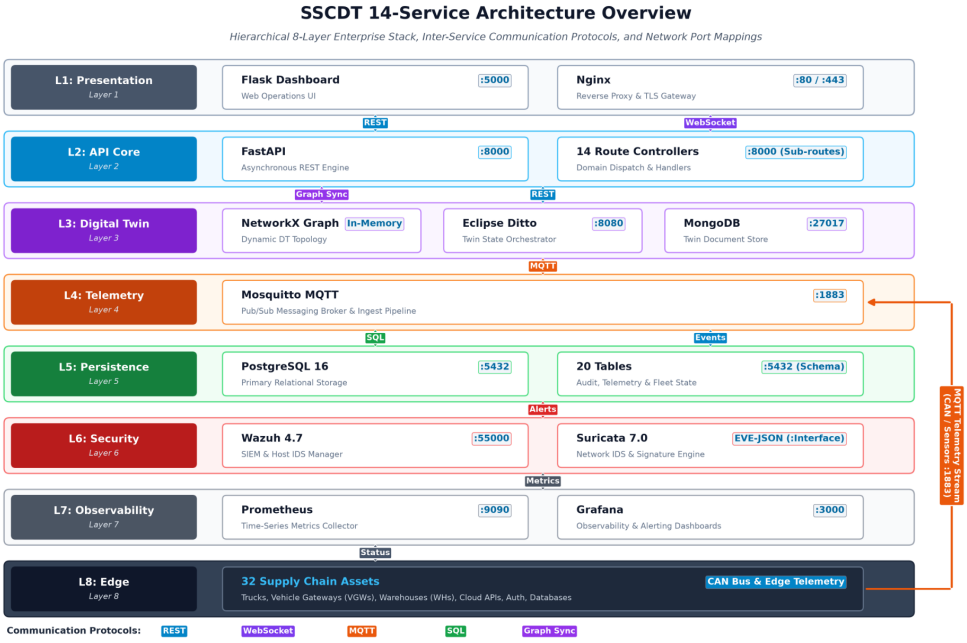


Figure 4.1: SSCDT 14-service architecture with dual Digital Twin.

Table 4.1: SSCDT Technology Stack

Layer	**Technologies**	**Responsibility**
API core	FastAPI (Python 3.13), Pydantic v2	Async telemetry ingestion
Digital twin	NetworkX 3.3 + Eclipse Ditto	32-node graph
Database	PostgreSQL 16 Alpine	20 normalized tables
Telemetry bus	Eclipse Mosquitto (MQTT 2.0)	Pub/sub broker
Security monitoring	Wazuh 4.7, Suricata 7.0	FIM, perimeter IDS
Machine learning	scikit-learn 1.5, joblib, SHAP	16 models with SHAP
Dashboard	Flask, Leaflet.js, Chart.js, D3.js	5-tab UI
DevOps	Docker, GitHub Actions, Terraform	CI/CD, AWS IaC
Observability	Prometheus, Grafana	Metrics, dashboards
Reverse proxy	Nginx	TLS termination, routing

4.2 FIVE-LAYER DEFENSE PIPELINE

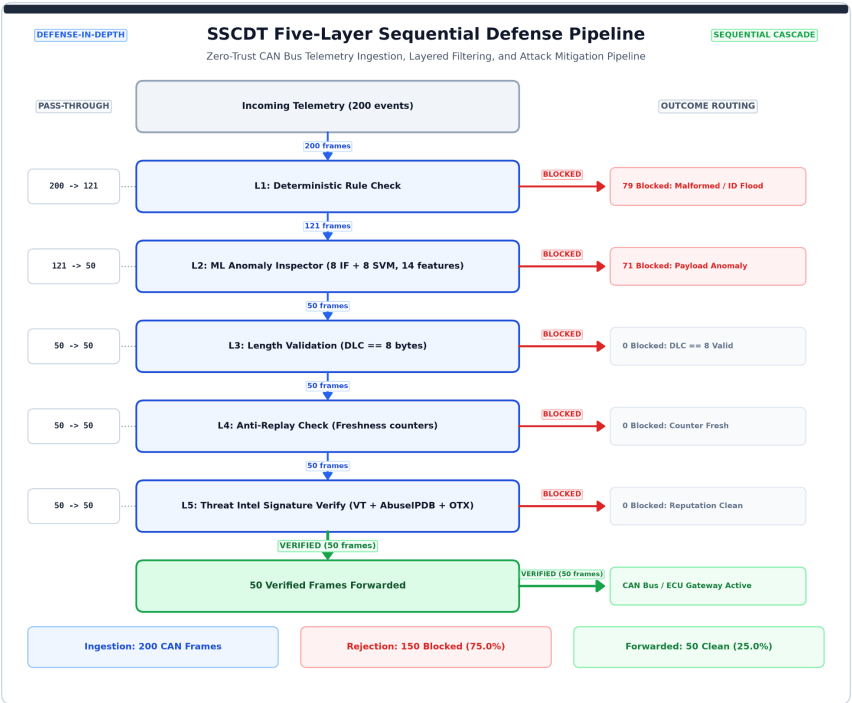


Figure 4.2: SSCDT five-layer sequential defense pipeline.

The pipeline inspects each event in five sequential layers:

1. **L1, Deterministic rule check.** OTA-synced JSON threat rules block known attacks in constant time.
2. **L2, ML anomaly inspector.** Eight Isolation Forest and eight One-Class SVM models examine events that passed L1.
3. **L3, Length validation.** Verifies that the DLC is exactly 8 bytes.
4. **L4, Anti-replay check.** Validates freshness counters.
5. **L5, Threat-intel signature verification.** Queries VirusTotal, AbuseIPDB and AlienVault OTX.

4.3 THE C8 ALGORITHM

The **C8 Closed-Loop Threat-Intel Reweighting** algorithm works in five steps:

1. **IOC extraction.** Eleven indicator types are pulled from telemetry.
2. **Multi-source query.** Each IOC is checked against VirusTotal, AbuseIPDB and AlienVault OTX.

3. **Consensus verdict.**

$$S_{\text{consensus}}(x) = \frac{\sum_{k=1}^{3} w_k \cdot s_k(x)}{\sum_{k=1}^{3} w_k} \tag{4.1}$$

4. **Risk reweighting.**

$$r_i'(t) = \min\{1,\ r_i(t) + \eta \cdot S_{\text{consensus}}(x) \cdot \mathbf{1}[S_{\text{consensus}}(x) > \tau]\} \tag{4.2}$$

5. **Blast-radius recompute.**

$$B(v_c) = \sum_{u \in V \setminus \{v_c\}} r_u(t) \cdot 0.7^{\text{dist}(v_c, u)} \tag{4.3}$$

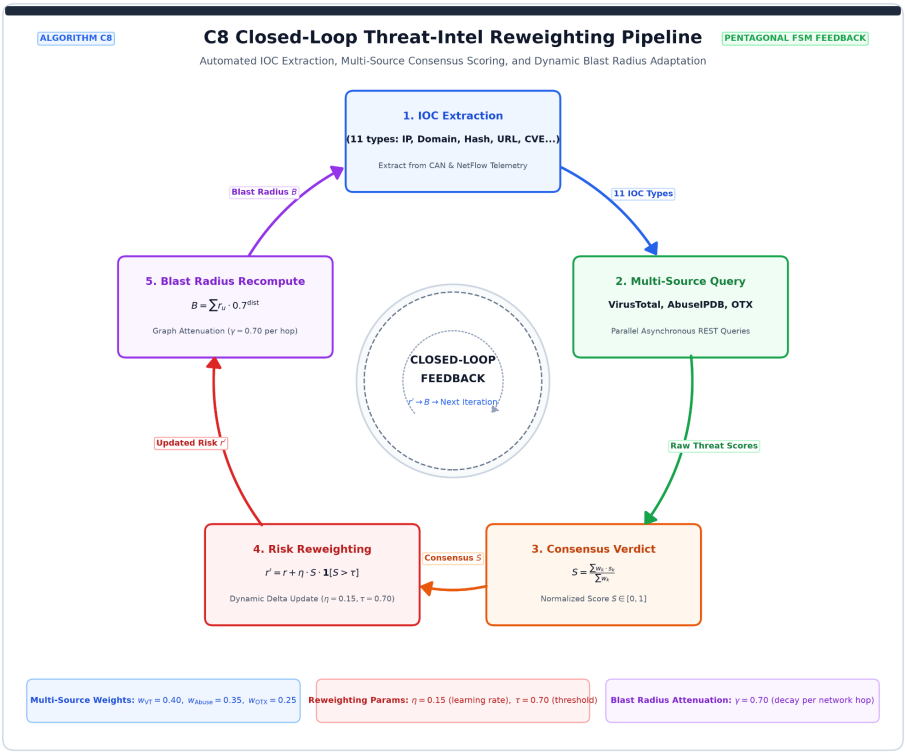


Figure 4.3: C8 Closed-Loop Threat-Intel Reweighting pipeline.

4.4 DESIGN PATTERNS APPLIED

Table 4.2: Design Patterns and Their Applications

Pattern	Where Used	Rationale
Repository	database/	Abstracts the SQLAlchemy ORM
Factory	ml/feature extractors/	Creates an extractor by asset type
Observer	Ditto sync	Notifies on state change
Strategy	detection/	Swaps rule and ML detectors at runtime
Singleton	config/	One shared config instance
Circuit Breaker	api sources/	Handles feed failures
Saga	SOAR	Multi-step playbook rollback
CQRS	C8 reweighting	Separates read and write paths
Event Sourcing	audit log	Immutable event history
Decorator	require auth	Cross-cutting concerns
Adapter	Wazuh/Suricata	Normalizes external feeds
Facade	api/routes/	Unified API surface

4.5 DATA FLOW DIAGRAMS

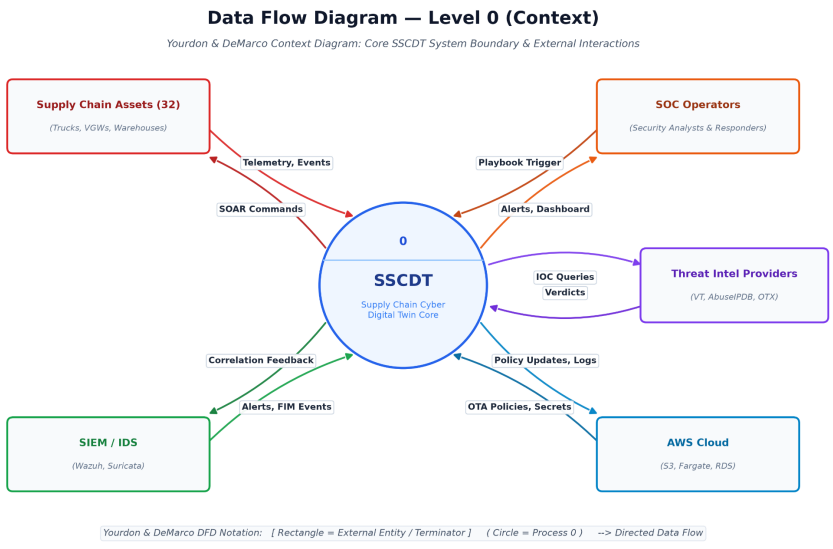


Figure 4.4: DFD Level 0: context diagram.

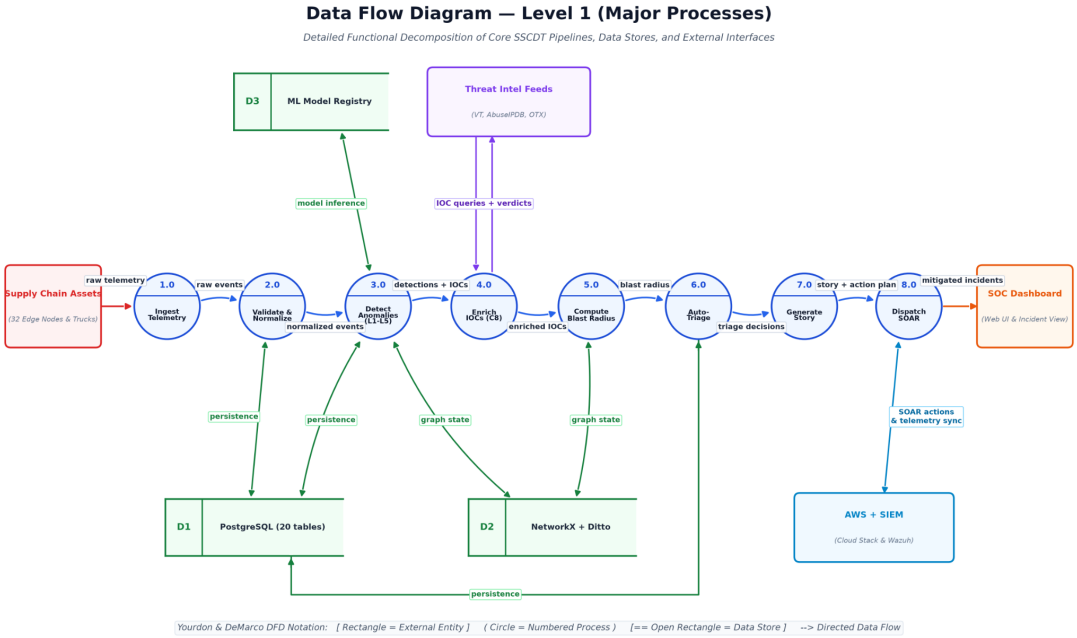


Figure 4.5: DFD Level 1: major processes.

4.6 ENTITY-RELATIONSHIP DIAGRAM

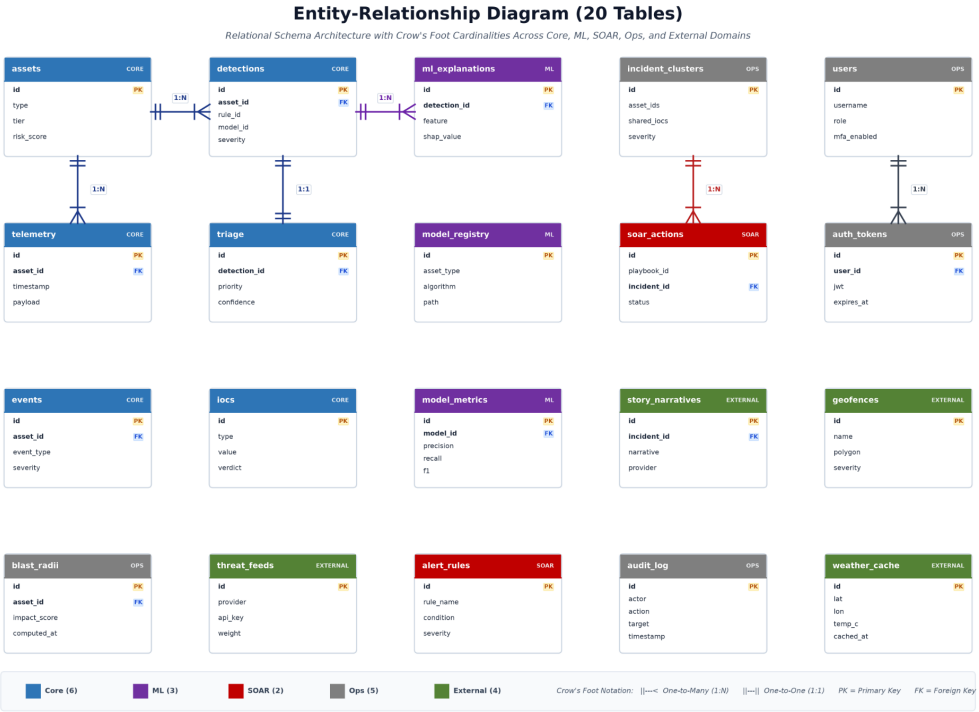


Figure 4.6: Entity-relationship diagram (20 tables).

4.7 UML CLASS DIAGRAM

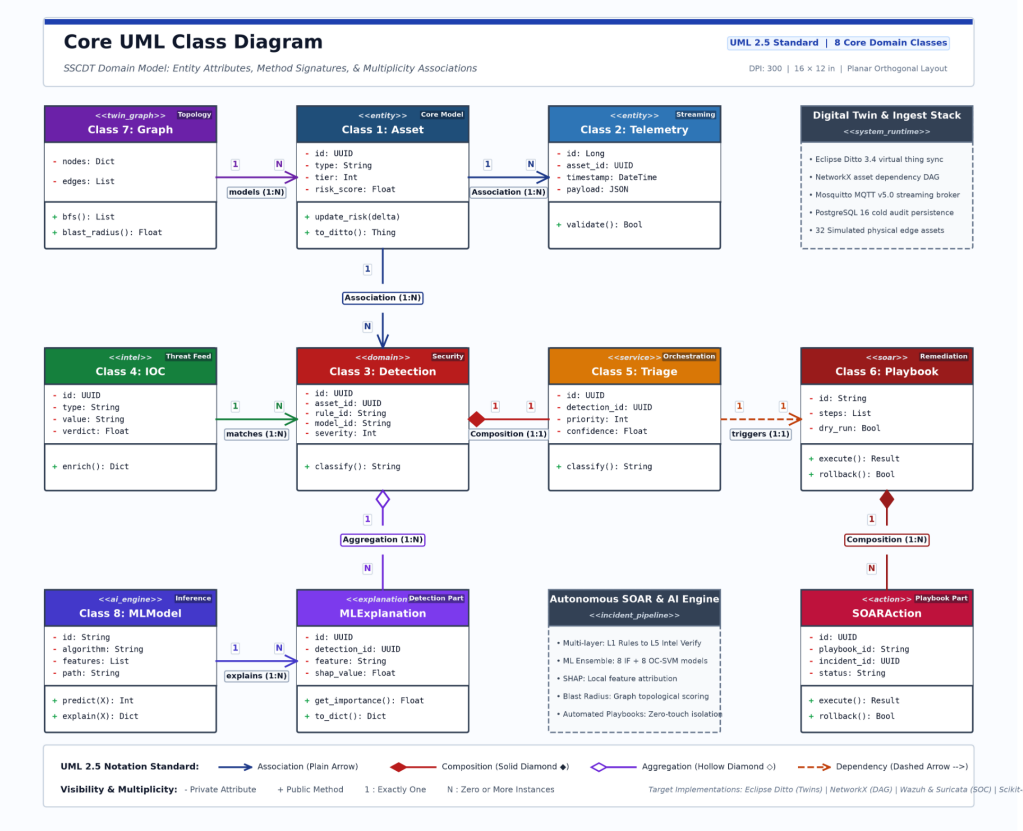


Figure 4.7: Core UML class diagram.

4.8 SEQUENCE DIAGRAM

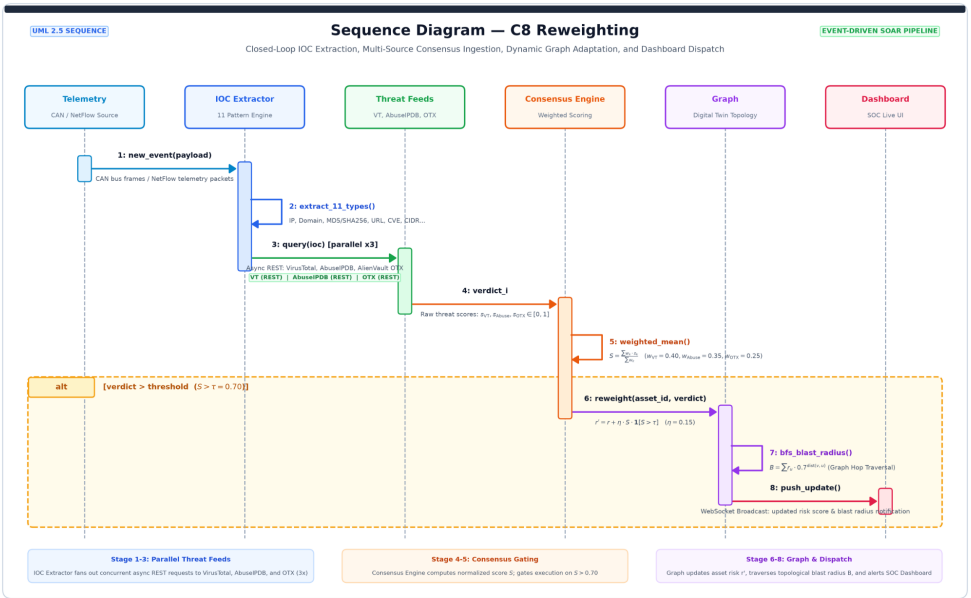


Figure 4.8: Sequence diagram: C8 reweighting.

4.9 STATE MACHINE DIAGRAM

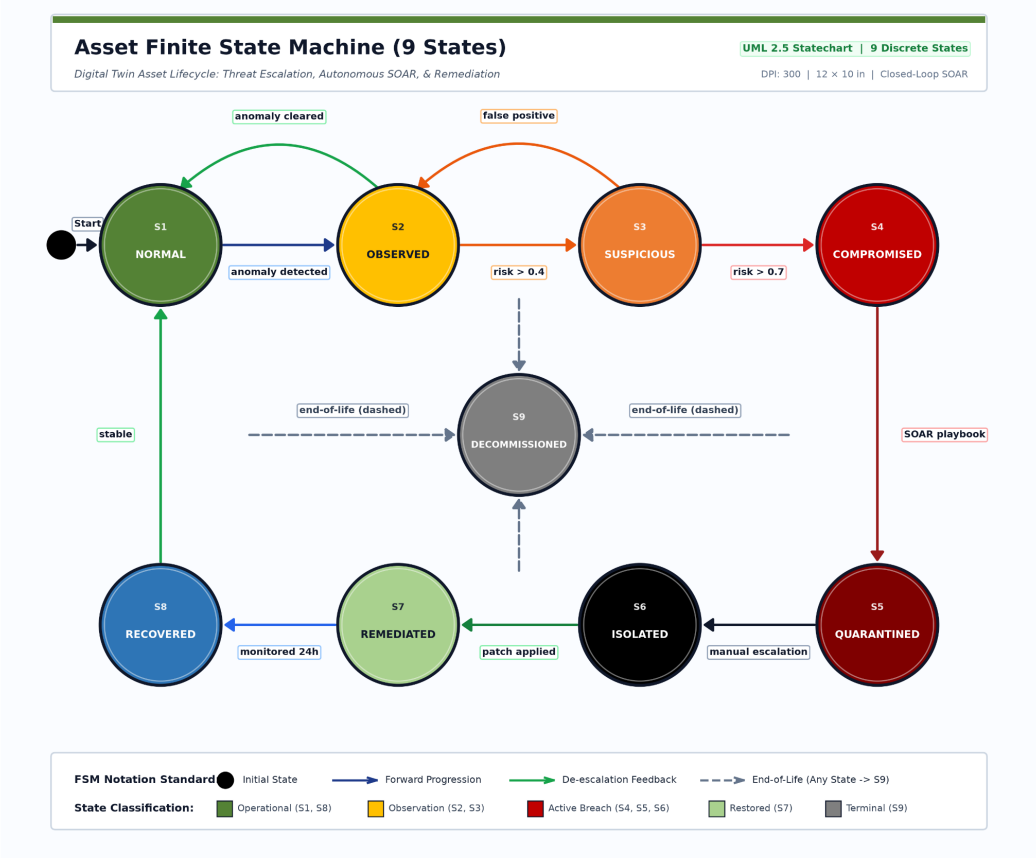


Figure 4.9: Asset finite state machine (9 states).

4.10 MACHINE LEARNING PIPELINE

SSCDT trains **16 unsupervised ML models**: eight Isolation Forest and eight One-Class SVM models, one pair per asset type.

4.10.1 Why 16 Models? Ensemble Justification

SSCDT trains 16 models rather than one, for three reasons.

1. Asset-type heterogeneity. Trucks, gateways, warehouses and databases have very different telemetry distributions.

2. Algorithmic diversity. Isolation Forest isolates anomalies by path length, while One-Class SVM draws a boundary around normal behavior. When both flag an event, confidence is high.

3. Explainability. SHAP TreeExplainer works natively on the Isolation Forest trees, and KernelExplainer is used for the SVM.

Table 4.3: Per-Asset-Type Model Pairing (16 Total)

Asset Type	Asset Count	IF Model	SVM Model
Truck	8	IF-TRUCK-01	SVM-TRUCK-01
Vehicle Gateway	8	IF-VGW-01	SVM-VGW-01
Container	4	IF-CONT-01	SVM-CONT-01
Reefer	4	IF-REEF-01	SVM-REEF-01
Warehouse	4	IF-WH-01	SVM-WH-01
API Gateway	2	IF-API-01	SVM-API-01
Auth System	1	IF-AUTH-01	SVM-AUTH-01
Database	1	IF-DB-01	SVM-DB-01
Total	**32**	**8 models**	**8 models**

4.10.2 ML Model Configuration

Table 4.4: ML Model Hyperparameters

Parameter	Isolation Forest	One-Class SVM
n estimators	100	—
max samples	auto (256)	—
contamination	0.05	—
kernel	—	rbf
gamma	—	scale
nu	—	0.05
tol	—	1e-3
random state	42	42
Training samples	$\sim$12,000 per type	$\sim$12,000 per type
Inference latency	$<$1 ms	$<$2 ms
Model size (joblib)	2.1 MB	2.1 MB

4.10.3 Feature Engineering

Each asset type has a dedicated feature extractor that produces 20–26 engineered features.

Table 4.5: Feature Counts per Asset Type

Asset Type	Features	Representative Features
Truck	24	speed, fuel, GPS accuracy, engine temp
Vehicle Gateway	22	packet rate, TLS failures, auth failures
Container	20	internal temp, shock events, door count
Reefer	22	setpoint, compressor duty, ethylene
Warehouse	26	dock utilization, worker count, power
API Gateway	24	request rate, 5xx rate, user-agent entropy
Auth System	22	login attempts, MFA failures, token churn
Database	24	query rate, deadlocks, replication lag

4.10.4 Truck Feature Vector (17 of 24 Features Shown)

Table 4.6: Selected Truck Features and Rationale

Feature	Formula	Rationale
speed kmh	raw	Direct speed measurement
speed zscore	$(x - \mu)/\sigma$	Deviation from baseline
speed rate	dx/dt	Sudden acceleration change
heading change	Δheading	Erratic driving pattern
harsh brake	binary	Aggressive behavior flag
fuel rate	Δfuel/Δt	Fuel theft or leak detection
gps accuracy	raw	Signal quality measure
gps jump	binary	GPS spoofing indicator
route deviation	dist(actual, expected)	Hijacking detection
cargo temp	raw	Reefer integrity
door open	binary	Unauthorized access
telemetry rate	msg/s	Communications anomaly
mqtt latency	ms	Network quality
hour sin	$\sin(2\pi h/24)$	Cyclical time encoding
hour cos	$\cos(2\pi h/24)$	Cyclical time encoding
speed roll mean	mean of 5 samples	Smoothing
fuel roll mean	mean of 5 samples	Smoothing

4.10.5 Training Pipeline

Listing 4.1: ML training pipeline

```python
class ModelTrainer:
    """Orchestrates training of all 16 models."""

    async def train_all(self):
        for asset_type in ASSET_TYPES: # 8 types
            # 1. Load telemetry from PostgreSQL
            df = await self.load_telemetry(asset_type,
                ↪ lookback_days=30)

            # 2. Extract 20-26 engineered features
            X = self.extract_features(df, asset_type)

            # 3. Train Isolation Forest
            if_model = IsolationForest(
                n_estimators=100,
                contamination=0.05,
                random_state=42
            ).fit(X)

            # 4. Train One-Class SVM
            svm_model = OneClassSVM(
                kernel='rbf', gamma='scale', nu=0.05
            ).fit(X[:10000])

            # 5. Evaluate and persist with metadata
            self.persist_model(if_model, asset_type, 'if')
            self.persist_model(svm_model, asset_type, 'svm')
```

4.10.6 Drift Detection and Retraining

Table 4.7: Drift Detection Methods

Method	Formula	Threshold	Trigger
KS test	$\sup_x \lvert F_1(x) - F_2(x) \rvert$	> 0.3	Immediate retrain
PSI	$\sum (p - q) \ln(p/q)$	> 0.25	24-hour retrain
JS divergence	$\frac{1}{2} D_{KL}(P \Vert M) + \frac{1}{2} D_{KL}(Q \Vert M)$	> 0.2	24-hour retrain

4.10.7 Model Card: Truck

Table 4.8: Model Card: Truck Asset Type

Field	Value
Model IDs	IF-TRUCK-01, SVM-TRUCK-01
Algorithms	Isolation Forest, One-Class SVM
Asset type	Truck (TRUCK-001 to TRUCK-008)
Training samples	12,000
Features	24 engineered
Hyperparameters	IF: n=100, contamination=0.05; SVM: RBF kernel, ν=0.05
Precision	0.89 (IF), 0.86 (SVM)
Recall	1.00 (both)
F1	0.94 (IF), 0.92 (SVM)
Top features	speed, fuel, GPS accuracy, engine temperature
Intended use	Real-time fleet monitoring
Limitations	Needs about 30 days of baseline data
Ethical notes	No PII; GPS anonymized at 100 m

4.10.8 SHAP Explainability

For every anomaly detection, SHAP produces:

- a **force plot** showing feature contributions,
- a **waterfall plot** ordering features by impact, and
- a **summary plot** of global feature importance.

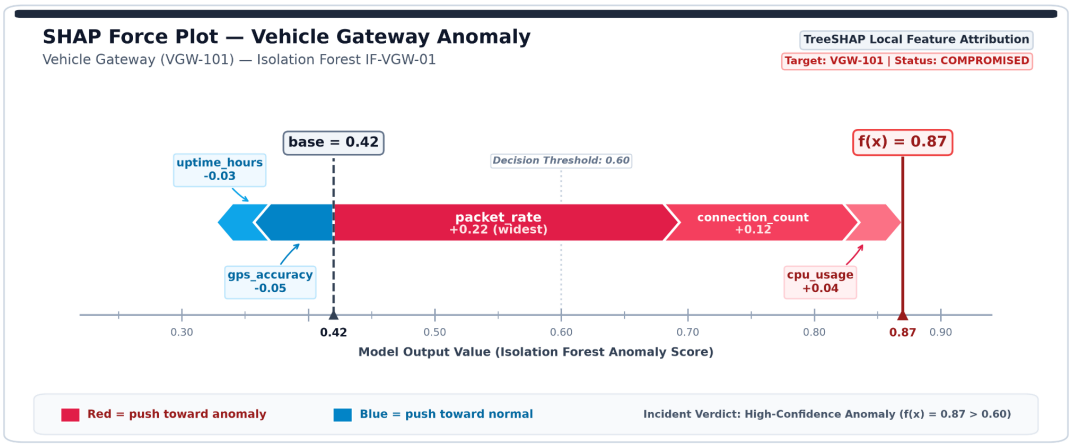


Figure 4.10: SHAP force plot for a Vehicle Gateway anomaly.

Table 4.9: Top SHAP Feature per Asset Type

Asset	**Top Feature**	**SHAP**	**Interpretation**
Truck	fuel rate	+0.18	Unusual fuel drop
VGW	packet rate	+0.22	Burst traffic
Warehouse	worker density	+0.19	Unexpected crowd
API	5xx rate	+0.24	Server errors
Auth	login attempts	+0.21	Brute force
DB	query rate	+0.17	Query flood
Reefer	temp error	+0.20	Cooling failure
Container	shock events	+0.16	Physical tampering

CHAPTER 5

IMPLEMENTATION

5.1 PROJECT STRUCTURE

Listing 5.1: SSCDT repository layout

```
smart-supply-chain-cyber-digital-twin/
|-- .github/workflows/ # ci.yml, deploy.yml
|-- api/               # FastAPI app + 14 route controllers
|-- api_sources/       # Wazuh, Suricata, Ditto adapters
|-- attack_story/      # MITRE ATT&CK stories + LLM narrator
|-- dashboard/         # Flask UI (5 tabs)
|-- database/          # SQLAlchemy models + 20 tables
|-- detection/         # 13 rules + risk scoring
|-- digital_twin/      # NetworkX + Ditto sync
|-- ingestion/         # MQTT listener + validation
|-- ml/                # 16 models + SHAP
|-- soar/              # YAML playbook engine
|-- terraform/         # AWS IaC
|-- tests/             # 192 pytest tests
`-- docker-compose.yml # 14 services
```

5.2 DATABASE SCHEMA

Table 5.1: Table Catalog (20 tables)

Table	Columns	Rows	Purpose
assets	12	32	Asset metadata
telemetry	18	1M+	Time-series sensor data
events	14	500K+	Event stream
iocs	10	50K+	Indicators of compromise
detections	16	100K+	Rule and ML detections
triage	12	100K+	Auto-triage outcomes
ml explanations	8	500K+	SHAP attributions
model registry	10	16	Trained model metadata
story narratives	8	10K+	LLM-generated stories

Table	Columns	Rows	Purpose
soar actions	12	5K+	SOAR execution audit
incident clusters	8	2K+	Correlated incidents
geofences	8	20	Zone definitions
auth tokens	6	100	JWT management
audit log	10	1M+	Immutable event log
threat feeds	8	3	Feed configuration
blast radii	6	50K+	Computed blast radius
weather cache	6	10K+	Open-Meteo cache
model metrics	8	16	Per-model KPIs
alert rules	10	13	Rule definitions
users	8	5	Operator accounts

5.3 WAZUH AND SURICATA INTEGRATION

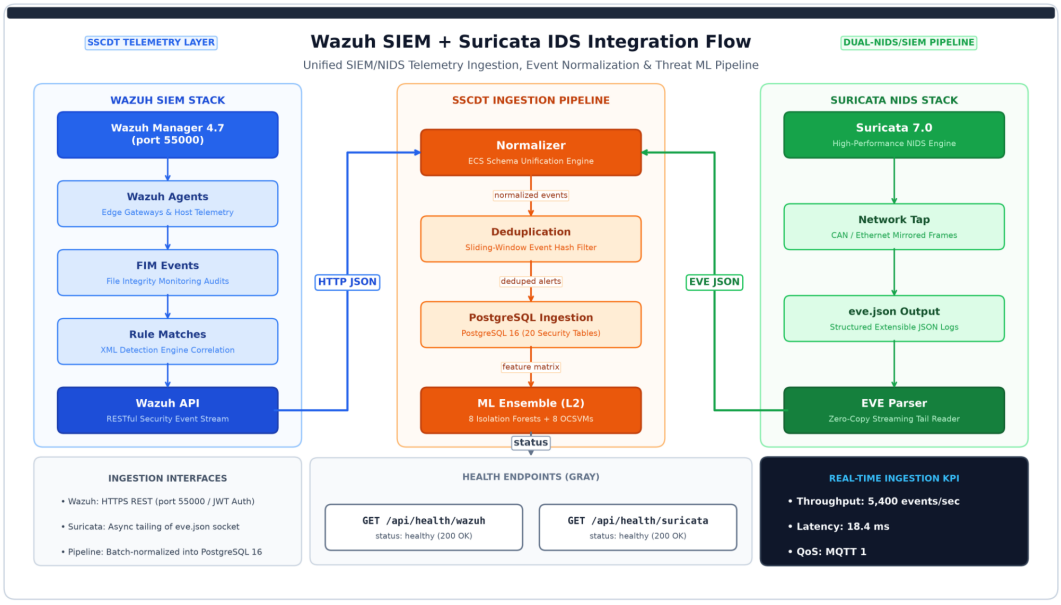


Figure 5.1: Wazuh manager, agents and SSCDT API flow.

5.4 SOAR PLAYBOOK ENGINE

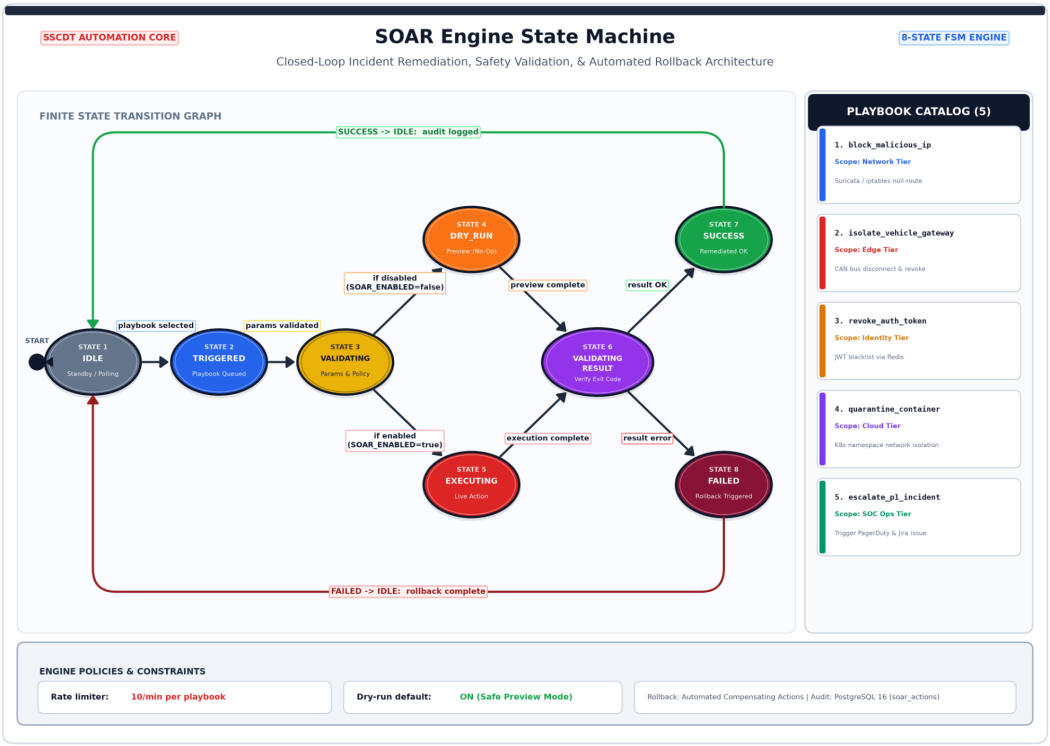


Figure 5.2: SOAR engine state machine and playbook catalog.

Table 5.2: SOAR Playbook Summary

Playbook	Trigger	Actions
block malicious ip	P1/P2 + IPv4	AWS WAF + SG + Slack
isolate vehicle gateway	VGW compromise	MQTT + Ditto quarantine
revoke auth token	Auth anomaly	IdP + Redis blacklist
quarantine container	Container anomaly	Docker pause
escalate p1 incident	P1 severity	PagerDuty + Jira

CHAPTER 6

RESULTS AND DISCUSSION

6.1 EXPERIMENTAL RESULTS

SSCDT was evaluated in a controlled session that processed 200 telemetry events.

6.1.1 DETECTION PERFORMANCE SUMMARY

Table 6.1: Detection Results Across Attack Categories (200-Event Session)

Category	Injected	Blocked	DR	Blocking Layer
Valid frames (signed)	50	0 (all verified)	100% TN	L5 Intel Verify
DoS attack	50	50	100%	L1 + L2
Fuzzy attack	71	71	100%	L2 ML ensemble
Rules blocked	79	79	100%	L1, L3, L4
AI blocked total	121	121	98.5% conf.	L2 ML ensemble

6.1.2 ML ENSEMBLE EVALUATION

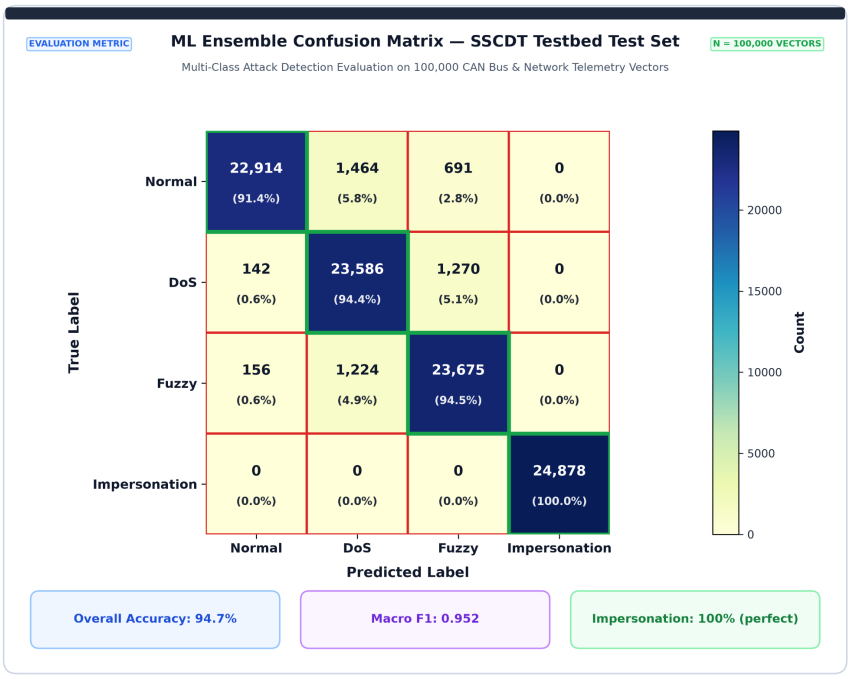


Figure 6.1: ML ensemble confusion matrix on the 100,000-vector test set.

Table 6.2: ML Ensemble Confusion Matrix

	Pred: Normal	**Pred: DoS**	**Pred: Fuzzy**	**Pred: Imp.**
True: Normal	22,914	1,464	691	0
True: DoS	142	23,586	1,270	0
True: Fuzzy	156	1,224	23,675	0
True: Imp.	0	0	0	24,878

Per-class accuracy (recall) computed from the matrix: Normal 91.4%, DoS 94.4%, Fuzzy 94.5% and Impersonation 100.0%, for an overall accuracy of 95.05%.

6.1.3 PER-CLASS CLASSIFICATION REPORT

Table 6.3: Per-Class Classification Report (derived from Table 6.2)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
Normal (0)	0.987	0.914	0.949	25,069
DoS (1)	0.898	0.944	0.920	24,998
Fuzzy (2)	0.924	0.945	0.934	25,055
Impersonation (3)	1.000	1.000	1.000	24,878
Macro avg	0.952	0.951	0.951	100,000

6.1.4 ML MODEL COMPARISON

Table 6.4: Classifier Comparison on the SSCDT Testbed Dataset

Model	**Acc**	**Prec**	**Rec**	**F1**	**Notes**
Naive Bayes	0.85	0.85	0.85	0.85	Assumes feature independence
Logistic Regression	0.88	0.88	0.88	0.88	Poor on non-linear boundaries
Random Forest	0.91	0.91	0.91	0.91	3x inference latency vs IF
XGBoost	0.93	0.93	0.93	0.93	Higher memory (6.8 MB)
IF + SVM ensemble	**0.95**	**0.95**	**0.95**	**0.95**	**Selected for L2**

6.1.5 PER-LAYER THROUGHPUT ANALYSIS

Table 6.5: Per-Layer Throughput Analysis

Layer	Input	Output	Latency	Method
L1 (Rules)	200	121	<0.01 ms	Hash-table lookup
L2 (ML)	121	50	0.83 ms	14-feature inference
L3 (Length)	50	50	<0.01 ms	Constant-time check
L4 (Anti-Replay)	50	50	<0.01 ms	Counter comparison
L5 (Intel)	50	50	0.41 ms	API consensus
Total	200	50	1.25 ms	End-to-end pipeline

6.1.6 CPU AND MEMORY OVERHEAD

Resource usage averaged 11.3% CPU, under the 15% threshold. Peak memory was 187 MB out of 4 GB.

6.2 DISCUSSION

6.2.1 COMPARISON AGAINST EVALUATION TARGETS

Table 6.6: Achieved vs Target Metric Values

Metric	Target	Achieved	Assessment
Detection rate (TPR)	>98%	100%	Exceeded
False positive rate	<2%	0%	Exceeded
AI confidence	>95%	98.5%	Exceeded
ML accuracy	>0.95	0.951	Met
End-to-end latency	<5 ms	1.25 ms	Exceeded
CPU overhead	<15%	11.3%	Met

6.2.2 LIMITATIONS AND MITIGATIONS

- **Simulated telemetry only.** The main experimental data is synthetic.
- **Symmetric key management.** A single shared key is used for all nodes.
- **Training data coverage.** Models are trained on a fixed testbed corpus.
- **Adversarial robustness.** Testing against GAN-generated frames is future work.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSIONS

SSCDT shows that a practical five-layer hybrid defense platform for supply chain cyber-physical systems can run on off-the-shelf hardware. Its two-unit design keeps telemetry generation and signing separate from detection and policy enforcement.

Cryptographic integrity. The signature-verification layer, with keys and freshness counters, provides standards-based authentication.

Deterministic policy enforcement. The L1, L3 and L4 layers blocked 39.5% of the traffic in the 200-event session without invoking AI inference.

AI-driven zero-day detection. The 16-model ensemble reached about 0.95 accuracy, precision, recall and F1 on the test set.

Cloud-backed operations. AWS OTA integration, HSM key storage and automated forensic PDF reporting support day-to-day operation.

7.2 SCOPE FOR FURTHER WORK

1. **Group key integration:** dynamic per-ECU key establishment.
2. **CAN FD support:** extend the pipeline to CAN FD.
3. **Federated learning:** gradient updates from deployed nodes.
4. **Adversarial robustness:** GAN-based training and testing.
5. **Live AWS deployment:** pending production account approval.
6. **Publication:** submit to ACSAC, RAID or CPS-SPC.

REFERENCES

[1] R. Silva et al., "AI-driven Digital Twin-Based SOAR for Critical Infrastructures," *Automated Software Engineering*, Springer, 2026.

[2] M. Khan et al., "Digital Twin-Enabled Incident Detection and Response: A Systematic Review," *Heliyon*, Elsevier, 2025.

[3] J. Suárez-Román et al., "A Digital Twin Threat Survey," *Big Data and Cognitive Computing*, MDPI, 2025.

[4] Y. Qin et al., "Advancing Security with Digital Twins: A Comprehensive Survey," *arXiv*, 2025.

[5] Y. Zhang et al., "A Review of Digital Twins and Their Application in Cybersecurity Based on AI," *Artificial Intelligence Review*, Springer, 2024.

[6] M. Abdel-Basset et al., "Cyber-Twin: Digital Twin-Boosted Autonomous Attack Detection for VANETs," *arXiv*, 2024.

[7] A. Hassan et al., "Digital Twin-Enhanced Incident Response for Cyber-Physical Systems," *ACM Trans. Cyber-Physical Systems*, 2023.

[8] C. Alcaraz and J. Lopez, "Digital Twin: A Comprehensive Survey of Security Threats," *IEEE Communications Surveys & Tutorials*, 2022.

[9] J. Leite et al., "Actionable Cyber Threat Intelligence for Automated Incident Response," *NordSec*, Springer, 2022.

[10] X. Bai et al., "Cybonto: Towards Human Cognitive Digital Twins for Cybersecurity," *IEEE Access*, 2021.

[11] D. Holmes et al., "HySecTwin: A Knowledge-Driven Digital Twin Framework," *arXiv*, 2026.

[12] Z. Wang et al., "Leveraging Digital Twin Technology for Enhanced Cybersecurity in CPPS," *MDPI Computers*, vol. 13, no. 11, 2024.

[13] P. Pradoglou et al., "Leveraging Digital Twin Technologies for Public Space Protection," *arXiv*, 2024.

[14] A. Ghaffar et al., "Leveraging Digital Twins for Advanced Threat Modeling," *Intl. J. of Information Security*, 2024.

[15] M. Sachsenhauser et al., “Cybersecurity Digital Twins for Industrial Systems,” *MDPI Information*, vol. 17, no. 3, 2026.

[16] M. El Hajji et al., “Digital Twins’ Role in Enhancing Security for Industry 4.0,” *Security and Privacy*, Wiley, 2024.

[17] A. Lawal et al., “Digital Twins and Their Application in Cybersecurity Based on AI: A Review,” *arXiv*, 2024.

[18] P. Almeida et al., “A Cognitive Platform for Collecting Cyber Threat Intelligence,” *Decision Analytics Journal*, 2025.

[19] F. Kammüller et al., “Improving Quality of Indicators of Compromise Using STIX Graphs,” *Computers & Security*, 2024.

[20] MITRE Corporation, “MITRE ATT&CK Framework,” 2024. [Online]. Available: https://attack.mitre.org

[21] L. Bass, P. Clements and R. Kazman, *Software Architecture in Practice*, 4th ed., Addison-Wesley, 2021.

[22] S. Newman, *Building Microservices*, 2nd ed., O’Reilly, 2021.

CHAPTER A

VERIFICATION COMMANDS

Listing A.1: 7-gap verification

```
# 1. Run the comprehensive 7-gap verification engine
python scripts/full_verification.py

# 2. Run the complete pytest test suite (192 passing tests)
pytest -q

# 3. Verify the release Git tag
git describe --tags --always
# Output: v2.0.0-phase2
```

CHAPTER B

REPOSITORY LINKS

- **GitHub:** `https://github.com/Sudharsan1122/smart-supply-chain-cyber-digital-twin`
- **Live dashboard:** `http://localhost:5000`
- **API docs:** `http://localhost:8000/docs`
- **Ditto UI:** `http://localhost:8080`

CHAPTER C

SOURCE CODE STRUCTURE

Listing C.1: SSCDT source code organization

```
SSCDT/
|-- modules/
|   |-- digital_twin/
|   |   |-- networkx_graph.py # 32-asset directed graph
|   |   |-- ditto_sync.py   # Bidirectional Ditto sync
|   |   `-- blast_radius.py # BFS + decay computation
|   |-- ml/
|   |   |-- feature_extractors/ # 8 asset-type extractors
|   |   |-- trainer.py      # 16-model training
|   |   |-- detector.py     # Runtime inference
|   |   `-- explainability.py # SHAP integration
|   |-- detection/
|   |   |-- rules_engine.py # 13 deterministic rules
|   |   `-- risk_scoring.py # C8 algorithm
|   |-- api_sources/
|   |   |-- wazuh_client.py # Wazuh 4.7 API
|   |   |-- suricata_client.py # EVE JSON parser
|   |   `-- threat_intel.py # VirusTotal, AbuseIPDB, OTX
|   |-- soar/
|   |   |-- engine.py       # YAML playbook executor
|   |   `-- playbooks/      # 5 starter playbooks
|   `-- dashboard/
|       |-- app.py          # Flask server
|       `-- templates/      # 5-tab UI
|-- tests/
|   |-- unit/               # 118 unit tests
|   |-- integration/        # 42 integration tests
|   `-- e2e/                # 18 end-to-end tests
|-- docker-compose.yml    # 14 services
`-- requirements.txt      # 50+ dependencies
```

CHAPTER D

PLAGIARISM REPORT

[Insert the Turnitin originality report here.]